

Lab_Assignment_2.2_Week_2

AI Assisted Coding - 2303A52077 - B37

Task 1: Cleaning Sensor Data

Task 1

```
def separate_numbers(numbers):
    if not numbers:
        print("The given list is empty")
        return [], []

    negative_numbers = [num for num in numbers if num < 0]
    non_negative_numbers = [num for num in numbers if num >= 0]

    return negative_numbers, non_negative_numbers

# Get user input for a list of numbers
user_input = input("Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): ")

# Convert the input string to a list of integers
try:
    original_numbers = [int(num) for num in user_input.split()]
except ValueError:
    print("Invalid input. Please enter numbers separated by spaces.")
    original_numbers = [] # Set to empty list if input is invalid

# Call the function to separate numbers
negatives, non_negatives = separate_numbers(original_numbers)

# Print results if the list was not empty
if original_numbers:
    print(f"Original list from user input: {original_numbers}")
    print(f"Separated negative numbers: {negatives}")
    print(f"New list with no negative numbers: {non_negatives}")

... Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): 1 -2 3 -4 0 -5 6
Original list from user input: [1, -2, 3, -4, 0, -5, 6]
Separated negative numbers: [-2, -4, -5]
New list with no negative numbers: [1, 3, 0, 6]
```

Input: [1, -2, 3, -4, 0, -5, 6]

Output: [1, 3, 0, 6]

PROMPT USED:

"write a python code to separate all the negative numbers from the user given list of numbers and show new list which contains no negative number start the code from scrap and if the list is empty give a statement or text saying the data is clean if the no say invalid data found"

Task 2: String Character Analysis

Code:

```
def count_chars(input_string):  
    vowels = "aeiou"  
    vowel_count = 0  
    consonant_count = 0  
    digit_count = 0  
    if not input_string:  
        print("Given String is empty")  
        return  
    for char in input_string:  
        char_lower = char.lower()  
        if 'a' <= char_lower <= 'z': # Check if it's an alphabet  
            if char_lower in vowels:  
                vowel_count += 1  
            else:  
                consonant_count += 1  
        elif '0' <= char <= '9': # Check if it's a digit
```

```
        digit_count += 1
print(f"Original String: {input_string}")
if vowel_count == 0:
    print("No vowels are found.")
else:
    print(f"Number of Vowels: {vowel_count}")
if consonant_count == 0:
    print("No consonants are found.")
else:
    print(f"Number of Consonants: {consonant_count}")
if digit_count == 0:
    print("No digits are found.")
else:
    print(f"Number of Digits: {digit_count}")
# Take input from the user
user_string = input("Enter a string: ")
# Call the function with the user's string
count_chars(user_string)
```

Prompt used:

"Write a Python function to count the number of vowels, consonants, and digits in a string. Use a sample input string and display the counts as output."

Input & output:

Enter a string: Thumma Vishal B37 2303a52077

Original String: Thumma Vishal B37 2303a52077

Number of Vowels: 5

Number of Consonants: 9

Number of Digits: 11

Working function:

```
def count_chars(input_string):
```

```
    vowels = "aeiou"
```

```
    vowel_count = 0
```

```
    consonant_count = 0
```

```
    digit_count = 0
```

```
    if not input_string:
```

```
        print("Given String is empty")
```

```
        return
```

```
    for char in input_string:
```

```
        char_lower = char.lower()
```

```
        if 'a' <= char_lower <= 'z': # Check if it's an alphabet
```

```
            if char_lower in vowels:
```

```
                vowel_count += 1
```

```
            else:
```

```
                consonant_count += 1
```

```
        elif '0' <= char <= '9': # Check if it's a digit
```

```
            digit_count += 1
```

```

def count_chars(input_string):
    vowels = "aeiou"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0

    if not input_string:
        print("Given String is empty")
        return

    for char in input_string:
        char_lower = char.lower()
        if 'a' <= char_lower <= 'z': # Check if it's an alphabet
            if char_lower in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
        elif '0' <= char <= '9': # Check if it's a digit
            digit_count += 1

    print(f"Original String: {input_string}")
    if vowel_count == 0:
        print("No vowels are found.")
    else:
        print(f"Number of Vowels: {vowel_count}")

    if consonant_count == 0:
        print("No consonants are found.")
    else:
        print(f"Number of Consonants: {consonant_count}")

    if digit_count == 0:
        print("No digits are found.")
    else:
        print(f"Number of Digits: {digit_count}")

```

```

# Take input from the user
user_string = input("Enter a string: ")

# Call the function with the user's string
count_chars(user_string)

```

```

... Enter a string: Thumma Vishal B37 2303a52077
Original String: Thumma Vishal B37 2303a52077
Number of Vowels: 5
Number of Consonants: 9
Number of Digits: 11

```

Task 3: Palindrome Check – Tool Comparison(Gemini/Copilot)

In Colab Gemini –

Prompt used – “write the palindrome checking function”

```
Task 3

[ ]

import re

def check_palindrome(input_string):
    # Remove non-alphanumeric characters and convert to lowercase
    processed_string = re.sub(r'^a-zA-Z0-9', '', input_string).lower()

    # Check if the processed string is equal to its reverse
    if processed_string == processed_string[::-1]:
        return "yes! given string is a palindrome"
    else:
        return "no! the given string is not a palindrome"

# Take input from the user
user_input_string = input("Enter a string to check if it's a palindrome: ")

# Call the function and print the result
result = check_palindrome(user_input_string)
print(result)

▼ Enter a string to check if it's a palindrome: vav
yes! given string is a palindrome
```

In Github Copilot – “#write a code that have palindrome-checking function def is_palindrome(s):”

```
first.py second.py X

second.py
1  #write a code that have palindrome-checking function def is_palindrome(s):
2  def is_palindrome(s):
3      # Remove spaces and convert to lowercase
4      s = s.replace(" ", "").lower()
5      # Check if the string is equal to its reverse
6      return s == s[::-1]
7  # Example usage
8  print(is_palindrome("A man a plan a canal Panama")) # True
9  print(is_palindrome("Hello")) # False
10 print(is_palindrome("racecar")) # True
11 print(is_palindrome("12321")) # True
12 print(is_palindrome("12345")) # False
13 #user-defined test cases
14 print(is_palindrome(input())) # Takes user input and checks if it's a palindrome
15
```

Task 4: Code Explanation Using AI

Task 3

01
✓

```
import re

def check_palindrome(input_string):
    # Remove non-alphanumeric characters and convert to lowercase
    processed_string = re.sub(r'[^a-zA-Z0-9]', '', input_string).lower()

    # Check if the processed string is equal to its reverse
    if processed_string == processed_string[::-1]:
        return "yes! given string is a palindrome"
    else:
        return "no! the given string is not a palindrome"

# Take input from the user
user_input_string = input("Enter a string to check if it's a palindrome: ")

# Call the function and print the result
result = check_palindrome(user_input_string)
print(result)
```

Enter a string to check if it's a palindrome: yes
yes! given string is a palindrome

Task 4

Gemini

1

explain the python function - check_palindrome, from the above code

+

Certainly! The `check_palindrome` function is designed to determine if a given input string is a palindrome. Here's a breakdown of how it works:

- Preprocessing:** It first cleans the input string by removing all non-alphanumeric characters (like spaces, punctuation, etc.) and converts the entire string to lowercase. This ensures that the comparison is case-insensitive and ignores irrelevant characters. It uses the `re.sub(r'[^a-zA-Z0-9]', '', input_string).lower()` method for this.
- Palindrome Check:** It then compares the processed string with its reverse. If the processed string is identical to its reversed version, it means the string is a palindrome.
- Return Value:** The function returns a descriptive string indicating whether the input string is a palindrome or not: "yes! given string is a palindrome" or "no! the given string is not a palindrome".

🗨️

test check_palindrome with a mixed-case string

Explain the 'user_input_string' variable in the file self

How to refactor the 'check_palindrome' function for efficiency?

What can I help you build?

+

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. 2/29/25



```
import re
```

```
def check_palindrome(input_string):
    # Remove non-alphanumeric characters and convert to lowercase
    processed_string = re.sub(r'[^a-zA-Z0-9]', '', input_string).lower()

    # Check if the processed string is equal to its reverse
    if processed_string == processed_string[::-1]:
        return "yes! given string is a palindrome"
    else:
        return "no! the given string is not a palindrome"

# Take input from the user
user_input_string = input("Enter a string to check if it's a palindrome: ")

# Call the function and print the result
result = check_palindrome(user_input_string)
print(result)
```

T explain the python function - `check_palindrome`, from the above code

◆ Certainly! The `check_palindrome` function is designed to determine if a given input string is a palindrome. Here's a breakdown of how it works:

1. **Preprocessing:** It first cleans the input string by removing all non-alphanumeric characters (like spaces, punctuation, etc.) and converts the entire string to lowercase. This ensures that the comparison is case-insensitive and ignores irrelevant characters. It uses the `re.sub(r'^a-zA-Z0-9','', input_string)` method for this.
2. **Palindrome Check:** It then compares the processed string with its reverse. If the processed string is identical to its reversed version, it means the string is a palindrome.
3. **Return Value:** The function returns a descriptive string indicating whether the input string is a palindrome or not: "yes! given string is a palindrome" or "no! the given string is not a palindrome".



Test `'check_palindrome'` with a mixed-case string

Explain the `'count_chars'` function in the first cell

How to refactor the `'check_palindrome'` function for efficiency

What can I help you build?



Gemini 2.5 Flash  


```

def multiply_list(numbers):
    product = 1
    for num in numbers:
        product *= num
    return product

my_numbers = [6, 7, 8, 9, 10]
result = multiply_list(my_numbers)

print(f"The list of numbers: {my_numbers}")
print(f"The product of the numbers is: {result}")

The list of numbers: [6, 7, 8, 9, 10]
The product of the numbers is: 30240

# Assuming the 'extract_negative_numbers' function is already defined in the environment.
# If not, please execute the cell where it is defined first (cell a641e4cd).

# Get user input for a list of numbers
user_input = input("Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): ")

# Convert the input string to a list of integers
try:
    user_numbers = [int(num) for num in user_input.split()]
except ValueError:
    print("Invalid input. Please enter numbers separated by spaces.")
    user_numbers = [] # Set to empty list if input is invalid

# Use the previously defined function to extract negative numbers
separated_negatives = extract_negative_numbers(user_numbers)

print(f"Original list from user input: {user_numbers}")
print(f"Separated negative numbers: {separated_negatives}")

Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): 1 -2 3 -4 0 -5 6
Original list from user input: [1, -2, 3, -4, 0, -5, 6]
Separated negative numbers: [-2, -4, -5]

def extract_negative_numbers(numbers):
    negative_numbers = []
    for num in numbers:
        if num < 0:
            negative_numbers.append(num)
    return negative_numbers

# Example usage:
my_list = [1, -2, 3, -4, 0, -5, 6]

```

```

negatives = extract_negative_numbers(my_list)

print(f"Original list: {my_list}")
print(f"Negative numbers: {negatives}")

Original list: [1, -2, 3, -4, 0, -5, 6]
Negative numbers: [-2, -4, -5]

# Get user input for a list of numbers
user_input = input("Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): ")

# Convert the input string to a list of integers
try:
    original_numbers = [int(num) for num in user_input.split()]
except ValueError:
    print("Invalid input. Please enter numbers separated by spaces.")
    original_numbers = [] # Set to empty list if input is invalid

if not original_numbers:
    print("The given list is empty")
else:
    negative_numbers = []
    non_negative_numbers = []

    for num in original_numbers:
        if num < 0:
            negative_numbers.append(num)
        else:
            non_negative_numbers.append(num)

    print(f"Original list from user input: {original_numbers}")
    print(f"Separated negative numbers: {negative_numbers}")
    print(f"New list with no negative numbers: {non_negative_numbers}")

Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6):
The given list is empty

```

Task 1

```

def separate_numbers(numbers):
    if not numbers:
        print("The given list is empty")
        return [], []

    negative_numbers = [num for num in numbers if num < 0]
    non_negative_numbers = [num for num in numbers if num >= 0]

    return negative_numbers, non_negative_numbers

```

```

# Get user input for a list of numbers
user_input = input("Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): ")

# Convert the input string to a list of integers
try:
    original_numbers = [int(num) for num in user_input.split()]
except ValueError:
    print("Invalid input. Please enter numbers separated by spaces.")
    original_numbers = [] # Set to empty list if input is invalid

# Call the function to separate numbers
negatives, non_negatives = separate_numbers(original_numbers)

# Print results if the list was not empty
if original_numbers:
    print(f"Original list from user input: {original_numbers}")
    print(f"Separated negative numbers: {negatives}")
    print(f"New list with no negative numbers: {non_negatives}")

Enter numbers separated by spaces (e.g., 1 -2 3 -4 0 -5 6): 1 -2 3 -4 0 -5 6
Original list from user input: [1, -2, 3, -4, 0, -5, 6]
Separated negative numbers: [-2, -4, -5]
New list with no negative numbers: [1, 3, 0, 6]

```

Task 2

```

def count_chars(input_string):
    vowels = "aeiou"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0

    if not input_string:
        print("Given String is empty")
        return

    for char in input_string:
        char_lower = char.lower()
        if 'a' <= char_lower <= 'z': # Check if it's an alphabet
            if char_lower in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
        elif '0' <= char <= '9': # Check if it's a digit
            digit_count += 1

    print(f"Original String: {input_string}")

```

```

if vowel_count == 0:
    print("No vowels are found.")
else:
    print(f"Number of Vowels: {vowel_count}")

if consonant_count == 0:
    print("No consonants are found.")
else:
    print(f"Number of Consonants: {consonant_count}")

if digit_count == 0:
    print("No digits are found.")
else:
    print(f"Number of Digits: {digit_count}")

# Take input from the user
user_string = input("Enter a string: ")

# Call the function with the user's string
count_chars(user_string)

Enter a string: Thumma Vishal B37 2303a52077
Original String: Thumma Vishal B37 2303a52077
Number of Vowels: 5
Number of Consonants: 9
Number of Digits: 11

```

Task 3

```

import re

def check_palindrome(input_string):
    # Remove non-alphanumeric characters and convert to lowercase
    processed_string = re.sub(r'[^\a-zA-Z0-9]', '',
input_string).lower()

    # Check if the processed string is equal to its reverse
    if processed_string == processed_string[::-1]:
        return "yes! given string is a palindrome"
    else:
        return "no! the given string is not a palindrome"

# Take input from the user
user_input_string = input("Enter a string to check if it's a
palindrome: ")

# Call the function and print the result
result = check_palindrome(user_input_string)
print(result)

```

```
Enter a string to check if it's a palindrome: vav  
yes! given string is a palindrome
```

Task 4